Kai Sheng

734-802-8898 | shengkai866@gmail.com | Ann Arbor, MI | linkedin.com/in/kai-sheng-044a27254 | shengkai866.github.io/personalWeb

EDUCATION

University of Michigan – Ann Arbor Sep 2026 – Dec 2027
Computer Science and Engineering (Master of Science), College of Engineering *Ann Arbor, MI*

University of Michigan – Ann Arbor Sep 2022 – May 2026
Computer Science (Bachelor of Science), College of Literature, Science, and the Arts *GPA: 3.85 / 4.00*

EXPERIENCE

Meituan May 2026 – Aug 2026
Software Development Engineer Intern, Data Platform *Beijing, China*

- Shipped an internal agent covering the PM-to-production data path end to end — knowledge Q&A, requirement discovery, metric validation, ETL delivery — replacing manual back-and-forth between PMs and data engineers.
- Team knowledge sat scattered across docs and IM threads, went stale, and contradicted itself; built a self-updating RAG knowledge base (BM25 + HNSW hybrid retrieval, cross-encoder reranking) and validated it with an LLM-driven eval pipeline — 100% retrieval routing on a 189-question sampled set (63 sources $\times$ 3), and 0 conflicts from a pairwise contradiction scan across all 2,300 segments.
- Halved the requirement handoff cycle ($\sim$2 days $\rightarrow$ $\sim$1 day) with a multi-turn elicitation workflow that turns a vague PM idea into an engineering ticket, clarifying metric definitions against existing dashboards and data assets and validating sample results up front.
- Reduced out-of-spec changes from $\sim$12% to $\sim$6% by gating automated ETL/DDL generation, platform testing, and validation queries behind engineer approval at four checkpoints — plan, metric definition, code, deploy.

University of Michigan – Ann Arbor Sep 2025 – Present
Research Assistant – Software Engineer · GLKB (Genomic Literature Knowledge Base) *glkb.org*

- Built the GraphRAG QA layer over the lab's 33M-article biomedical knowledge graph, pairing Faiss ANN search across 100M sentence embeddings with Neo4j traversal over 627M relationships, so answers follow entity relationships, not keyword matches, and every claim carries an inline citation.
- LLM agents couldn't query the graph without unbounded traversals; designed a REST tool API — entity retrieval, literature search, relation/ontology queries, subgraph extraction — exposing bounded graph operations, with async I/O so concurrent traversals don't block.
- Cut per-interaction latency from 1,596 ms to 445 ms (72% faster) with KV caching that reuses context across multi-agent turns, plus a Redis layer so repeated questions skip retrieval and generation entirely.
- Split compute from API to keep agent inference warm under spiky traffic: Lambda authenticates, rate-limits, and autoscales per request at the edge, forwarding to a long-lived EC2 service that avoids model reload.

PROJECT EXPERIENCE

Small-to-Large Model Prefill for Faster LLM Inference Oct 2026 – Present
Cross-model KV-cache projection + selective recompute *PyTorch, Transformers, Qwen3, SLURM*

- Prefill dominates time-to-first-token (TTFT) on long prompts; designed a hybrid prefill where Qwen3-0.6B reads the prompt and a trained 86M-param projector predicts the residual stream of every layer of a frozen Qwen3-8B, which the 8B's own norms, k/v projections and RoPE turn into an in-distribution KV cache; the 8B computes only the first and last few tokens.
- Projected KV alone lost exact retrieval; added selective recompute — the small model's attention to the question picks the top 10% of sentences for the 8B to recompute in place within the same forward call — lifting RULER at 8k from 0.64 to 0.85 (8B alone: 0.92).
- Trained only the projector with both LLMs frozen: residual-stream alignment, then KL distillation on 8B self-generated responses with gradients through the frozen 8B; a mixed data pool (QA, code, chat, math, retrieval) raised doc-QA F1 from 0.19 (web-only) to 0.69.
- Reached $\sim$2$\times$ TTFT speedup at 4k–8k while keeping $\sim$92% of the 8B's RULER score and matching its GSM8K (0.92 vs 0.91); benchmarked against a reimplemented SpecPrefill (ICML 2025), and cut QA fabrication from 13.7% to 5.3% vs. token dropping.

Distributed System Search Engine Aug 2025 – Dec 2025
Two-stage retrieval + Learning-to-Rank *Python, Hadoop, React, BERT*

- Engineered an end-to-end distributed search engine — text preprocessing, index construction, two-stage retrieval, and Learning-to-Rank behind a React frontend and Flask REST API — to surface documents that are semantically relevant but share no query terms.
- Indexing a 200K-document corpus exceeded single-machine capacity; built an offline pipeline in Python and Hadoop MapReduce that processes the full corpus and emits sharded inverted indexes.
- Generated Top-10 latent user queries per document with BERT as index-time expansion to match how users actually search.
- Trained a LambdaMART Learning-to-Rank model on 13,555 labeled samples over TF-IDF, BM25, bi-encoder and cross-encoder relevance plus PageRank/HITS graph features, re-ranking the BM25 + bi-encoder top 100 — NDCG@10 $\approx$ 0.51, MAP@10 $\approx$ 0.36, above TF-IDF/BM25 baselines.

SKILLS

Languages: Python, SQL, JavaScript, Java, C++, R, HTML/CSS
AI / ML: LangChain, LangGraph, BERT/Transformers, PyTorch, scikit-learn, Pandas, NumPy
Frameworks & Cloud: Flask, FastAPI, React, Spark, Hadoop; AWS (Lambda, EC2, Amplify)
Databases: Neo4j, PostgreSQL, MySQL, MongoDB, Hive, Oracle